RoslynGate

El puente semántico determinista para agentes de IA en .NET.

Este documento define el propósito, la arquitectura, las herramientas expuestas, los casos de uso, la gobernanza del agente y la estrategia de distribución de RoslynGate: un servidor MCP nativo que conecta a los agentes con la tabla de símbolos del compilador en lugar de con búsquedas de texto.

Dejar de adivinar con texto plano

RoslynGate es un servidor MCP (Model Context Protocol) ultraligero y agnóstico a la plataforma, compilado en C# con Native AOT, diseñado para erradicar por completo el uso de búsquedas de texto plano — `grep`, `ripgrep`, expresiones regulares— por parte de agentes de IA como Claude Code o Cursor en bases de código .NET.

Al conectar directamente al agente con el árbol sintáctico y la tabla de símbolos del compilador mediante JSON-RPC sobre `stdio`, se eliminan las alucinaciones de sintaxis, los falsos positivos y el desperdicio masivo de tokens de contexto.

1 El problema

El agente busca con texto: encuentra comentarios, strings y homónimos; no entiende sobrecargas, herencia ni genéricos. Cada intento fallido consume contexto y produce respuestas plausibles pero falsas.

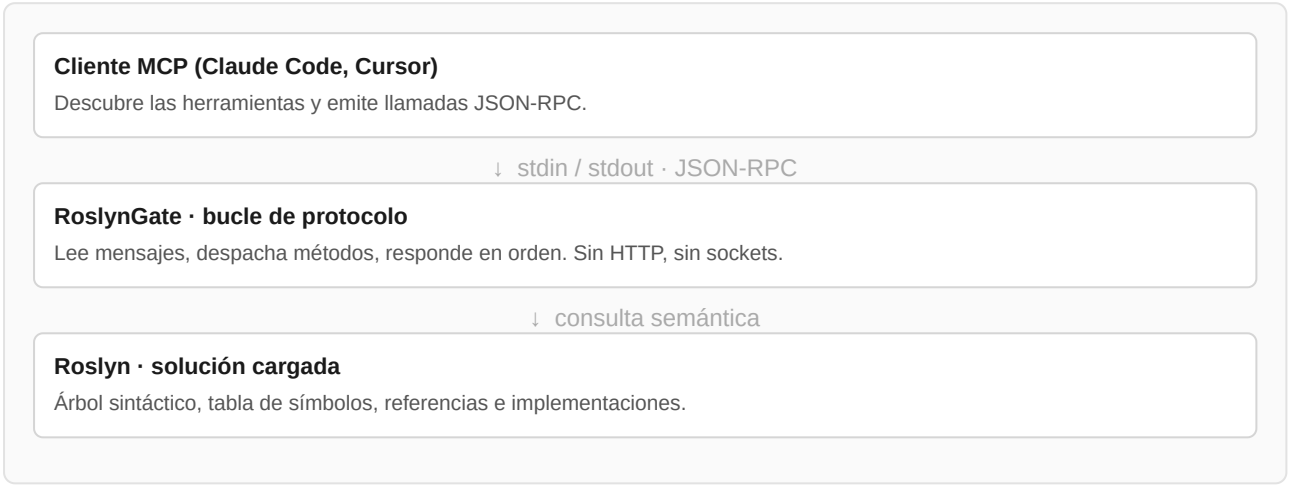
2 La solución

Una consulta semántica devuelve solo lo que el compilador conoce: firma exacta, referencias reales y jerarquía de tipos. Menos tokens, cero ambigüedad, resultado determinista.

> *El agente no necesita leer más código: necesita preguntarle al compilador exactamente lo que el compilador ya sabe.*

Un proceso hijo invisible

RoslynGate no es un servicio que se despliega: es un binario que el cliente MCP lanza como proceso hijo y controla por tuberías estándar.



Transporte

Comunicación bidireccional asíncrona por `stdio`, el estándar de MCP.

Sin frameworks

Cero ASP.NET Core, cero HTTP, cero WebSockets. Nada que infle el arranque.

Nativo

Native AOT para arranque en milisegundos y uso de RAM mínimo.

Por qué importa el arranque

El agente lanza el servidor cada vez que inicia una sesión. Un servidor que tarda segundos en arrancar convierte cada conversación en una espera perceptible; uno que arranca en milisegundos se vuelve invisible.

Tres funciones, ningún ruido

Para mantener la máxima velocidad y evitar la saturación de la ventana de contexto del agente, el servidor expone exclusivamente tres funciones deterministas.

HERRAMIENTA	RESPONDE	SALIDA
get_symbol_definition	¿Qué es exactamente este símbolo?	Firma exacta, modificadores, tipo de retorno y metadatos tomados del nodo AST del compilador.
find_symbol_references	¿Dónde se usa realmente?	Referencias cruzadas, sobrecargas, herencia e implementaciones de interfaces en toda la solución, sin falsos positivos de texto.
get_type_hierarchy	¿De qué hereda y qué contrato cumple?	Relaciones de herencia de clases base y contratos de interfaces, de forma instantánea.

El principio de diseño

Cada herramienta responde una pregunta que un agente no puede contestar leyendo texto: el compilador resuelve el símbolo, no el modelo. Tres herramientas bien definidas son más baratas de descubrir y de invocar que una API amplia que obliga al agente a elegir.

DETERMINISTA · SIN FALSOS POSITIVOS · BAJO CONSUMO DE TOKENS · SUPERFICIE MÍNIMA

Dos modos para encontrar la solución

Para garantizar flexibilidad absoluta sin romper flujos de trabajo en máquinas o estructuras de carpetas distintas, RoslynGate admite dos modos de resolución del archivo de solución (`.sln`).

Automático por autodetección

Sin argumentos, el servidor analiza el directorio de trabajo actual heredado del proceso del agente y busca de forma determinista el archivo de solución local.

- Ideal para repositorios de un solo proyecto.
- Sin configuración por máquina.
- La regla de búsqueda debe ser explícita y estable.

Explícito por parámetro

Si se indica una ruta en la configuración del cliente MCP, el servidor omite la búsqueda automática y carga directamente la solución indicada.

- Ideal para monorepos y soluciones múltiples.
- Elimina la ambigüedad del directorio actual.
- El error de ruta se reporta, no se adivina.

Regla de precedencia

El argumento explícito siempre gana. La autodetección es una comodidad, nunca una suposición silenciosa: si hay cero o más de una solución candidata, el servidor falla con un mensaje que nombra las rutas encontradas.

El freno de mano

Para evitar que el LLM recurra a la opción perezosa de usar texto plano, la distribución incluye directivas obligatorias de configuración para el entorno del agente (`.clauderc` / `.cursorrules`).

1 Prohibición léxica

Queda prohibido usar `grep` , `ripgrep` o `find` sobre archivos de código fuente C# para responder consultas de símbolos.

2 Obligación contractual

Toda consulta de símbolos, referencias o jerarquía se enruta exclusivamente a través de las herramientas de RoslynGate.

Por qué una directiva y no solo una herramienta

Una herramienta disponible compite con el hábito. La directiva convierte la ruta semántica en la ruta por defecto: el agente no elige entre buscar y preguntar, porque buscar deja de ser una opción válida dentro del repositorio.

> *La herramienta correcta no basta si el agente sigue prefiriendo el atajo equivocado.*

SIN GREP EN C# · SIN RIPGREP · SIN REGEX SOBRE CÓDIGO · CONSULTA SEMÁNTICA OBLIGATORIA

06 — CASOS DE USO

Dónde el puente cambia el trabajo

Los cuatro escenarios siguientes comparten una condición: el agente debe responder con precisión sobre código que no puede leer completo sin gastar el contexto de la sesión.

1 Onboarding en un repo heredado

Un agente entra a una solución de 40 proyectos y necesita saber dónde se implementa `IPaymentGateway`. En lugar de recorrer archivos con texto, pide la jerarquía y obtiene las implementaciones reales, con su archivo y su firma.

2 Refactor de una firma pública

Antes de cambiar `ChargeAsync`, el agente localiza todas las llamadas, sobrecargas e implementaciones afectadas. El cambio se hace con evidencia, no con una lista de coincidencias aproximadas.

3 Revisión de arquitectura

Un tech lead quiere introducir una abstracción nueva. El agente mapea la jerarquía existente, detecta implementaciones huérfanas y confirma qué contratos ya cubren el caso antes de escribir código.

4 Monorepo multi-solución

Cada agente se configura con la ruta explícita de su solución. Las consultas quedan aisladas por workspace, sin que la autodetección elija la solución equivocada del repositorio.

El patrón común

En los cuatro casos la pregunta es la misma —¿qué es este símbolo y quién lo usa?— y la respuesta correcta solo existe en la tabla de símbolos. La diferencia no es la velocidad de búsqueda: es que una coincidencia de texto no distingue una llamada real de un comentario, un string o un método homónimo de otra clase.

> *El agente deja de leer para adivinar y empieza a preguntar para saber.*

Cuatro fases hasta el binario distribuible

FASE	FOCO	ENTREGABLE
1	Bucle base y protocolo	Lector y escritor de mensajes JSON-RPC sobre `stdio`, más el manejo de los métodos de inicialización de MCP.
2	Motor semántico y workspace	Cargador de soluciones con autodetección y ruta explícita, conectado a las APIs de consulta de Roslyn.
3	Optimización Native AOT	Depuración de dependencias incompatibles, compilación estática y medición de arranque por debajo de 50 ms.
4	Empaquetado y distribución	Manifiestos de integración para clientes MCP y empaquetado multiplataforma.

Un riesgo que la fase 3 debe resolver

La carga de soluciones con MSBuildWorkspace y el objetivo Native AOT tensionan entre sí: el workspace depende de MSBuild y de carga dinámica, mientras que AOT recorta y prohíbe reflexión dinámica. La fase 3 no es solo "optimizar": es donde se decide si el binario usa el workspace completo o un cargador de metadatos compatible con AOT. Esa decisión debe tomarse con una medición, no con una suposición.

Descargar y conectar en un paso

1 Binarios nativos

Compilaciones estáticas autocontenidas y optimizadas para Windows, Linux y macOS, publicadas en GitHub Releases y listas para descarga directa. Sin SDK de .NET en la máquina destino.

2 Wrapper npm

Un paquete ligero para `npx` / `bunx` que detecta el sistema operativo, descarga el ejecutable correspondiente y lo vincula al archivo de configuración del cliente MCP.

Matriz de entrega

CANAL	USUARIO OBJETIVO	COSTO DE ADOPCIÓN
GitHub Releases	Equipos que fijan versiones y auditan binarios	Descargar y apuntar la configuración
Wrapper npm	Uso individual y CI	Una línea en la configuración MCP

Criterio de éxito

Un desarrollador debe poder pasar de "no tengo nada" a "mi agente consulta la tabla de símbolos" sin compilar C#, sin instalar el SDK y sin editar manualmente la configuración del cliente.

GITHUB RELEASES · WINDOWS · LINUX · MACOS · NPX / BUNX · JSON-RPC SOBRE STDIO